```
*****************************
    factorial_con_vector.c
*****************************
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#define MAX 20


void *factorial(void *arg)
{
  int i, n, fact;

  n = *(int *)arg;
  // printf("Parámetro leído: %d\n", n);

  fact=1;
  if (n < 2)
  {   printf("Hilo %lu: Factorial de %d: %d\n",pthread_self(), n, fact);
      pthread_exit(0);
  }

  for (i=2; i<=n; i++)
    fact = fact * i;
  printf("Hilo %lu: Factorial de %d: %d\n",pthread_self(), n, fact);
  pthread_exit(0);
}

int main(int argc, char * argv[])
{
  int i, n, n_hilos, v[MAX];

  pthread_t  hilo[MAX];

  if (argc==1) {printf("Error: Has de introducir el número de hilos\n"); exit(-1);}

  n_hilos=argc-1;

  for (i=0;i<n_hilos;i++)
  {
      v[i] = atoi(argv[i+1]);
      pthread_create(&hilo[i],NULL,&factorial,(void *)&v[i]);
  }

  for (i=0;i<n_hilos;i++)
      pthread_join(hilo[i],NULL);

  pthread_exit(0);
}

*****************************
    factorial_con_argv.c
*****************************
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#define MAX 20


void *factorial(void *arg)
{
  int i, n, fact;

  n = atoi((char *)arg);
  // printf("Parámetro leído: %d\n", n);
```

```
  fact=1;
  if (n < 2)
  {   printf("Hilo %lu: Factorial de %d: %d\n",pthread_self(), n, fact);
      pthread_exit(0);
  }

  for (i=2; i<=n; i++)
    fact = fact * i;
  printf("Hilo %lu: Factorial de %d: %d\n",pthread_self(), n, fact);
  pthread_exit(0);
}

int main(int argc, char * argv[])
{
  int i, n, n_hilos;

  pthread_t  hilo[MAX];

  if (argc==1) {printf("Error: Has de introducir el número de hilos\n"); exit(-1);}

  n_hilos=argc-1;

  for (i=0;i<n_hilos;i++)
  {
      n=atoi(argv[i+1]);
      pthread_create(&hilo[i],NULL,&factorial,(void *)argv[i+1]);
  }

  for (i=0;i<n_hilos;i++)
      pthread_join(hilo[i],NULL);

  pthread_exit(0);
}

**********************
    factorial.c --> Ojo! Genera warnings
**********************
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#define MAX 20

void *factorial(void *arg)
{
  int i, n, fact;

  n = (int)arg;
  // printf("Parámetro leído: %d\n", n);

  fact=1;
  if (n < 2)
  {   printf("Hilo %lu: Factorial de %d: %d\n",pthread_self(), n, fact);
      pthread_exit(0);
  }

  for (i=2; i<=n; i++)
    fact = fact * i;
  printf("Hilo %lu: Factorial de %d: %d\n",pthread_self(), n, fact);
  pthread_exit(0);
}

int main(int argc, char * argv[])
{
  int i, n, n_hilos;

  pthread_t  hilo[MAX];
```

```
    if (argc==1) {printf("Error: Has de introducir el número de hilos\n"); exit(-1);}

    n_hilos=argc-1;

    for (i=0;i<n_hilos;i++)
    {
        n=atoi(argv[i+1]);
        pthread_create(&hilo[i],NULL,&factorial,(void *)n);
    }

    for (i=0;i<n_hilos;i++)
        pthread_join(hilo[i],NULL);

    pthread_exit(0);
  }



  **********************
      factorial_resul_en_exit.c
  **********************
  #include <pthread.h>
  #include <stdio.h>
  #include <unistd.h>
  #include <stdlib.h>

  #define MAX 20

  int n_hilos;

  void *factorial(void *arg)
  {
    int i, n, *fact;

    n = *(int *)arg;
    //printf("Parámetro leído: %d\n", n);

    fact = (int *) malloc(sizeof(int));
    /* Habría que comprobar si malloc falla y devuelve NULL */

    *fact=1;

    if (n < 2) pthread_exit((void *) fact);

    for (i=2; i<=n; i++)
      *fact = *fact * i;
    //printf("Hilo %lu: Factorial de %d: %d\n",pthread_self(), n, *fact);
    pthread_exit((void *) fact);
  }

  int main(int argc, char * argv[])
  {
    int i, v[MAX], n, *fact;

    pthread_t  hilo[MAX];

    if (argc==1) {printf("Error: Has de introducir el número de hilos\n"); exit(-1);}

    n_hilos = argc - 1;

    for (i=0;i<n_hilos;i++)
    {
        v[i] = atoi(argv[i+1]);
        pthread_create(&hilo[i],NULL,&factorial,(void *)&v[i]);
    }

    for (i=0;i<n_hilos;i++)
    {
```

```
        pthread_join(hilo[i],(void *) &fact);
        n=atoi(argv[i+1]);
        printf("Factorial de %d: %d\n", n, (int)*fact);
        free(fact);
    }

    pthread_exit(0);
}


**********************
    genera_vector_aleatorio.c
**********************
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

#define MAX 20

int main()
{ time_t t;
  int i, v[MAX];

  srandom(time(&t));    // Genera semilla para no crear siempre
                        // los msimos números aleatorios
  for (i=0;i<MAX;i++)
  { v[i]=random()%10;  // Crea un número aleatorio entre 0 y 9
    printf("V[%d]=%d\n",i,v[i]);
  }
  exit(0);
}


**********************
    pot2.c
**********************
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

#define MAX 20

void *pot2(void *arg)
{
  int i, n;
  int *pot;

  n = *(int *) arg;
  //printf("Parámetro leído: %d\n", n);

  pot = (int *) malloc(sizeof(int));
  /* Habría que comprobar si malloc falla y devuelve NULL */

  *pot = 1;
  if (n == 0) pthread_exit((void *) pot);

  for (i=1; i<=n; i++)
    *pot = *pot * 2;

  pthread_exit((void *) pot);
}

int main(int argc, char * argv[])
{
  int i, n, n_hilos, v[MAX], e, *pot;

  pthread_t  hilo[MAX];
```

```
    if (argc==1) {printf("Error: Has de introducir el número de hilos\n"); exit(-1);}

    n_hilos=atoi(argv[1]);

    for (i=0;i<n_hilos;i++){
        v[i] = i;
        pthread_create(&hilo[i],NULL,&pot2,(void *)&v[i]);
    }

    for (i=0;i<n_hilos;i++)
    {
        pthread_join(hilo[i],(void *) &pot);
        printf("2 ^ %d: %d\n", i, *pot);
        free(pot);
    }

    pthread_exit(0);
  }


  ********************
      buscar_num.c
  ********************
  #include <pthread.h>
  #include <stdio.h>
  #include <unistd.h>
  #include <stdlib.h>
  #include <time.h>

  #define MAX 20

  int n_a_buscar, pos=-1;
  int v[MAX];

  typedef struct {
    int iini;
    int ifin;
  } parametros;

  void *busca_hilo(void *arg)
  {
    int i, ini, fin;
    parametros * argum;

    argum=(parametros *)arg;

    ini=argum->iini;
    fin=argum->ifin;

    for (i=ini; i<=fin; i++)
    {
      if (pos != -1) break;
      if (v[i] == n_a_buscar)
      {
        pos=i;    // OJO! Podría fallar si varios hilos acceden simultáneamente a la var. global
  "pos"
        printf("****Hilo %lu: Encuentra el número en V[%d]\n",pthread_self(),i);
        break;
      }
    }
    pthread_exit(NULL);
  }

  int main(int argc, char * argv[])
  {
    time_t t;
    int i,j;
    int n_hilos, dim_subvector;
```

```
  pthread_t  hilo[MAX];
  parametros * argum;

  n_a_buscar = atoi(argv[1]);
  n_hilos    = atoi(argv[2]);
  dim_subvector=MAX / n_hilos;

  if (n_a_buscar > 10)
  {  printf("El número a buscar ha de ser menor o igual que 10\n");
     pthread_exit(0);
  }

  if (n_hilos > MAX)
  {  printf("El número de hilos ha de ser menor o igual que 20\n");
     pthread_exit(0);
  }

  srandom(time(&t));

  printf("VECTOR\n");
  for (i=0;i<MAX;i++)
  {
    v[i]=random()%10;
    printf("V[%d]=%d\n",i,v[i]);
  }
  printf("\n");
  for (j=0;j<(n_hilos-1);j++)
    {
      argum=(parametros *)malloc(sizeof(argum));
      argum->iini=j*dim_subvector;
      argum->ifin=((argum->iini) + dim_subvector -1);
      printf("Hilo %d: Buscará entre las componenetes %d y %d\n", j, argum->iini, argum->ifin);

      pthread_create(&hilo[j],NULL,busca_hilo,(void *)argum);
    }
  /* Último hilo: procesa el último fragmento del vector */
  j=n_hilos-1;
  argum=(parametros *)malloc(sizeof(argum));
  argum->iini=j*dim_subvector;
  argum->ifin=(MAX-1);
  printf("Hilo %d: Buscará entre las componenetes %d y %d\n", j, argum->iini, argum->ifin);
  pthread_create(&hilo[j],NULL,busca_hilo,(void *)argum);

  for (i=0;i<n_hilos;i++)
      pthread_join(hilo[i],NULL);
  if (pos != -1)
       printf("El número %d está en la posición %d (V[%d]=%d)\n",n_a_buscar,pos,pos,v[pos]);
     else
        printf("El número %d no está en el vector\n",n_a_buscar);

  pthread_exit(0);
}



**************************
    suma_intervalo_sol.c
**************************
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMTHREADS 4

typedef struct interval{
   int iter;
   int ini;
   int fin;
```

```
} t_interval;

int suma_parcial[NUMTHREADS];
int suma = 0;

void *hilo(void *arg)
{ int  i;
  int ini1,fin1, itera;
  t_interval *inter;

  inter = ((t_interval *) (arg));
  itera = inter->iter;
  ini1 = inter->ini;
  fin1 = inter->fin;

  suma_parcial[itera] = 0;
  for(i=ini1;i<=fin1;i++){
   suma_parcial[itera] = suma_parcial[itera] + i;
  }
  printf("Hilo i=%d, suma = %d\n",itera,suma_parcial[itera]);
  pthread_exit(0);
}

int main()
{
   pthread_t thread[NUMTHREADS];
   int ini,fin;
   int i,elem,inicio;
   t_interval param[NUMTHREADS];

   printf("Introduce inicio intervalo: ");
   scanf("%d",&ini);
   printf("Introduce fin intervalo: ");
   scanf("%d",&fin);

   elem = ((fin-ini)+1)/NUMTHREADS;

   inicio=ini;
   for (i=0;i<NUMTHREADS;i++)
   {
     param[i].iter = i;
     param[i].ini = inicio;
     if (i==(NUMTHREADS-1)) param[i].fin = fin;
     else param[i].fin = param[i].ini + elem;

     pthread_create(&thread[i], NULL, hilo, (void *) &param[i]);
     printf("Hilo 1 iter = %d, ini = %d, fin = %d\n",param[i].iter,param[i].ini,param[i].fin);
     inicio = param[i].fin+1;
  }
   for (i=0; i<NUMTHREADS; i++) pthread_join(thread[i],NULL);

   for (i=0; i<NUMTHREADS; i++) suma = suma + suma_parcial[i];

   printf("Suma = %d\n ",suma);
   pthread_exit (0);
}



**************************
    max_vector_sol.c
**************************
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>

#define MAX_NUMTHREADS 20   /* We create MAX_NUMTHREADS at most */
```

```
#define VSIZE 2000000       /* Problem size */

int num_threads, subvector_dim;
int v[VSIZE];

typedef struct {
  int begin;
  int end;
  int nthread;
} arguments;

void *f_max(void *arg)
{
  int i, begin_v, end_v, nthread, *max_for_thread;
  arguments * argum;

  argum=(arguments *)arg;

  begin_v=argum->begin;
  end_v=argum->end;
  nthread=argum->nthread;

  max_for_thread = (int *) malloc(sizeof(int));
  /* We should test than malloc runs ok */

  *max_for_thread=0;
  for (i=begin_v; i<=end_v; i++)
    if (v[i]>*max_for_thread) *max_for_thread=v[i];
  printf("Maximum computed by thread %02d (%lu):  %08d\n", nthread, pthread_self(),
*max_for_thread);
  pthread_exit((void *)max_for_thread);
}

int main(int argc, char * argv[])
{
  int i, max, * partial_max;
  time_t t;

  pthread_t  thread[MAX_NUMTHREADS];
  arguments * argum;

  pthread_attr_t attr;

  pthread_attr_init(&attr); /* They are to be joinable */
  pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);

  srandom(time(&t));

  if (argc==1) {printf("Error: The number of threads is required as an argument\n"); exit(-1);}

  num_threads=atoi(argv[1]);


  for (i=0; i<VSIZE; i++)  /* We create the vector */
    v[i] = random() % VSIZE;
  puts("Data vector created");


  subvector_dim=VSIZE / num_threads;
  for (i = 0; i < num_threads; i++) /* We create threads, it can fail */
    {
      argum=(arguments *)malloc(sizeof(argum));
      argum->begin=i*subvector_dim;
      if ( i == (num_threads-1) )  argum->end=VSIZE-1;
      else                         argum->end=((argum->begin) + subvector_dim -1);
      argum->nthread=i;

      if  ( pthread_create(&thread[i], &attr, f_max, (void *)argum))
      {  printf("On iteration %d: ", i);
```

```
           perror("error creating thread.");
           exit(-1);
        }
      }
   printf("%d Threads successfully created.\n\n", i);


   max=0;
   for (i = 0; i < num_threads; i++) { /* We wait for their completion, it can fail */
                                       /* and get the maximum element returned by threads */
        if  ( pthread_join(thread[i],(void *)&partial_max))
       {  printf("On iteration %d: ", i);
          perror("error joining thread.");
          exit(-1);
       }
       if ( *partial_max > max ) max=*partial_max;
   }
   printf("\n%d Threads successfully joined.\n\n", i);


   printf("Final maximum:  %d\n", max);


   pthread_attr_destroy(&attr);    /* To release memory */
   exit(0);
 }
```